

Documentație Proiect: Nmap Professional Visualizer

1. Introducere și Cerințe Funcționale

Introducere

Proiectul **Nmap Professional Visualizer** este o aplicație web de tip Single Page Application (SPA) concepută pentru a simplifica și eficientiza analiza rapoartelor de securitate generate de utilitarul Nmap. Scanările de rețea produc adesea fișiere XML masive și greu de citit în format brut. Această soluție oferă o interfață grafică intuitivă, capabilă să proceseze complet local fișiere de până la 50MB, extrăgând și structurând datele vitale: adrese IP, porturi deschise, sisteme de operare și detalii despre servicii.

Scopul principal este de a oferi analiștilor de securitate (sau administratorilor de rețea) un instrument rapid, sigur (fără a trimite date sensibile către servere externe) și ușor de utilizat.

Cerințe Funcționale

Pe baza implementării curente, aplicația îndeplinește următoarele cerințe funcționale:

- **CF1. Încărcarea fișierelor:** Sistemul trebuie să permită încărcarea fișierelor XML prin mecanisme de tip "Drag & Drop" sau prin selecție manuală.
- **CF2. Procesare asincronă:** Sistemul trebuie să analizeze fișierele în fundal, afișând un indicator de încărcare, pentru a nu bloca interfața utilizatorului (UI).
- **CF3. Afișarea metadatelor:** Aplicația va extrage și afișa informațiile generale ale scanării (versiunea Nmap, comanda executată, data scanării).
- **CF4. Vizualizare tabelară și Paginare:** Host-urile detectate vor fi afișate într-un tabel structurat, divizat în pagini (ex: 20 de elemente per pagină) pentru o navigare ușoară.
- **CF5. Funcție de căutare (Filtrare):** Sistemul va permite căutarea în timp real a host-urilor folosind criterii precum adresa IP, numele (hostname) sau producătorul (vendor).
- **CF6. Inspecție detaliată:** Utilizatorul va putea vizualiza detalii complete despre un anumit host (toate porturile, protocoalele, starea și versiunile serviciilor) prin intermediul unei ferestre modale.

2. Uneltele utilizate

Pentru dezvoltarea acestui proiect, s-a optat pentru o abordare *Vanilla* (fără framework-uri externe grele) pentru a garanta performanța maximă și o amprentă de memorie redusă.

- **HTML5:** A fost utilizat pentru scheletul aplicației și definirea zonelor semantice (header, zone de drop, tabele, ferestre modale). Au fost folosite elemente specifice precum `<input type="file" hidden>` pentru ascunderea mecanismelor standard și înlocuirea lor cu o zonă de drop personalizată.
- **CSS3 (Custom Properties & Flexbox/Grid):**
 - **Variable CSS (:root):** Utilizate pentru a menține o paletă de culori consistentă (tematică profesională: nuanțe de albastru, slate, și culori semantice pentru statusuri).
 - **Flexbox și Grid:** Folosite pentru aranjarea elementelor de bord, alinierea butoanelor de paginare și structurarea metadatelor scanării pe multiple coloane responsive.
 - **Tranziții și Animații:** Implementate pentru feedback-ul vizual la interacțiunile utilizatorului (ex: animația spinner-ului de încărcare, efectele de hover pe zona de drop).
- **JavaScript (ES6+) & Web Workers API:**
 - **Vanilla JS:** Controlează manipularea DOM-ului (afișarea/ascunderea modalelor, generarea dinamică a rândurilor de tabel).
 - **Web Workers:** Reprezintă un element cheie al uneltelor utilizate. Web Worker-ul permite delegarea unui script JavaScript pentru a rula în fundal, independent de thread-ul principal. Aceasta previne înghețarea interfeței în timpul procesării string-urilor XML foarte mari.
- **Expresii Regulate (RegEx):** Deoarece API-ul DOMParser nu este disponibil în interiorul unui Web Worker nativ, s-a folosit un set complex de expresii regulate pentru a extrage eficient informațiile din nodurile `<host>` și `<port>`. Această abordare este extrem de rapidă comparativ cu conversia întregului fișier într-un arbore DOM.
- **Git & GitHub / GitLab:** <https://github.com/alixic/Nmap-Pro-Analyzer>

3. Descrierea arhitecturii proiectului

Arhitectura proiectului Nmap Professional Visualizer urmează un model strict **Client-Side Processing (fără Backend)**. Decizia de a elimina serverul backend a fost luată din motive de securitate a datelor: rezultatele scanărilor de rețea conțin adesea informații sensibile despre

topologia rețelei, vulnerabilități și IP-uri interne. Prin urmare, arhitectura asigură că datele nu părăsesc niciodată browser-ul utilizatorului.

Arhitectura poate fi împărțită în trei straturi (layers) principale:

A. Stratul de Prezentare (UI Layer)

Acest strat este gestionat de HTML și CSS și este responsabil pentru interacțiunea directă cu utilizatorul. Acesta este format din trei stări principale ale interfeței:

1. **Starea de Upload:** Afișează zona de *Drag & Drop*. Folosește un FileReader API pentru a prelua conținutul fișierului de pe disc.
2. **Starea de Procesare:** Ascunde zona de upload și afișează un *Loader* (#loader). În acest moment, interfața grafică așteaptă semnalul de la stratul de procesare, rămânând fluidă și responsivă.
3. **Starea Dashboard:** Afișează tabelul principal, bara de căutare, paginarea și datele scanării. Componentele sunt generate dinamic folosind innerHTML pe baza datelor structurate primite.

B. Stratul Logic (Main Thread Logic Layer)

Acest strat acționează ca un "Controller" în cadrul browser-ului. El interceptează evenimentele din UI (click-uri, input de la tastatură, schimbarea paginii) și actualizează stratul de prezentare.

- **State Management:** Menține starea aplicației folosind variabile globale simple: allHosts (sursa de adevăr cu toate datele), filteredHosts (datele rezultate după căutare), și currentPage (starea paginării).
- **Mecanismul de Filtrare și Paginare:** Funcția renderPage() calculează indicii de început și de sfârșit pe baza listei filteredHosts și afișează doar un sub-set de 20 de elemente, optimizând astfel operațiunile pe DOM și reducând consumul de memorie în browser.

C. Stratul de Procesare a Datelor (Background Worker Layer)

Acesta este nucleul aplicației din punct de vedere arhitectural.

- **Inițializarea:** La selecția fișierului, interfața creează un Web Worker "on-the-fly" utilizând Blob și URL.createObjectURL(). Acest lucru permite ca tot codul worker-ului să fie inclus în același fișier HTML, eliminând necesitatea unor fișiere externe și facilitând portabilitatea proiectului.
- **Fluxul de Date (Data Flow):**

1. Șirul de caractere (string-ul XML) este trimis de la *Main Thread* către *Worker* prin `worker.postMessage()`.
2. În interiorul worker-ului, un motor de parsare bazat pe expresii regulate izolează nodurile `<host>`. Pentru fiecare host, se extrag IP-ul, MAC-ul, vendor-ul, numele și sistemul de operare.
3. Ulterior, se caută nodurile `<port>` în interiorul fiecărui host, extrăgând starea, protocolul, numele serviciului și versiunea combinată a produsului.
4. Worker-ul construiește un obiect JSON structurat ce conține datele metadate (`scanInfo`) și array-ul de host-uri (`hosts`).
5. Prin `self.postMessage()`, obiectul JSON este trimis înapoi către *Main Thread*, unde stratul logic îl preia, actualizează starea și apelează funcția de randare.

Această separare arhitecturală garantează că, indiferent cât de complex și de mare este fișierul XML încărcat, interfața utilizatorului nu va suferi de fenomene de tip *freeze* (blocaj), respectând astfel principiile moderne de dezvoltare web.

4. Diagrama Entitate Relație

Structura datelor extrase din fișierul XML și salvate în memoria aplicației formează un set clar de entități relationate.

Notă: Deoarece aplicația procesează date în memorie (JSON) și nu folosește o bază de date relațională, această diagramă reflectă structura obiectelor din memorie.

Fragment de cod

xerDiagram

```
SCANINFO ||--o{ HOST : "conține rezultate pentru"
```

```
HOST ||--o{ PORT : "are deschise/filtrate"
```

```
SCANINFO {
```

```
    string version "Versiune Nmap"
```

```
    string args "Comanda executată"
```

```
    string date "Data și ora scanării"
```

```
}
```

```
HOST {  
  
    string ip "Adresa IPv4 (Primary Key logic)"  
  
    string mac "Adresa MAC"  
  
    string vendor "Producător echipament"  
  
    string hostname "Numele în rețea"  
  
    string status "Stare (up/down)"  
  
    string os "Sistem de operare detectat"  
  
}
```

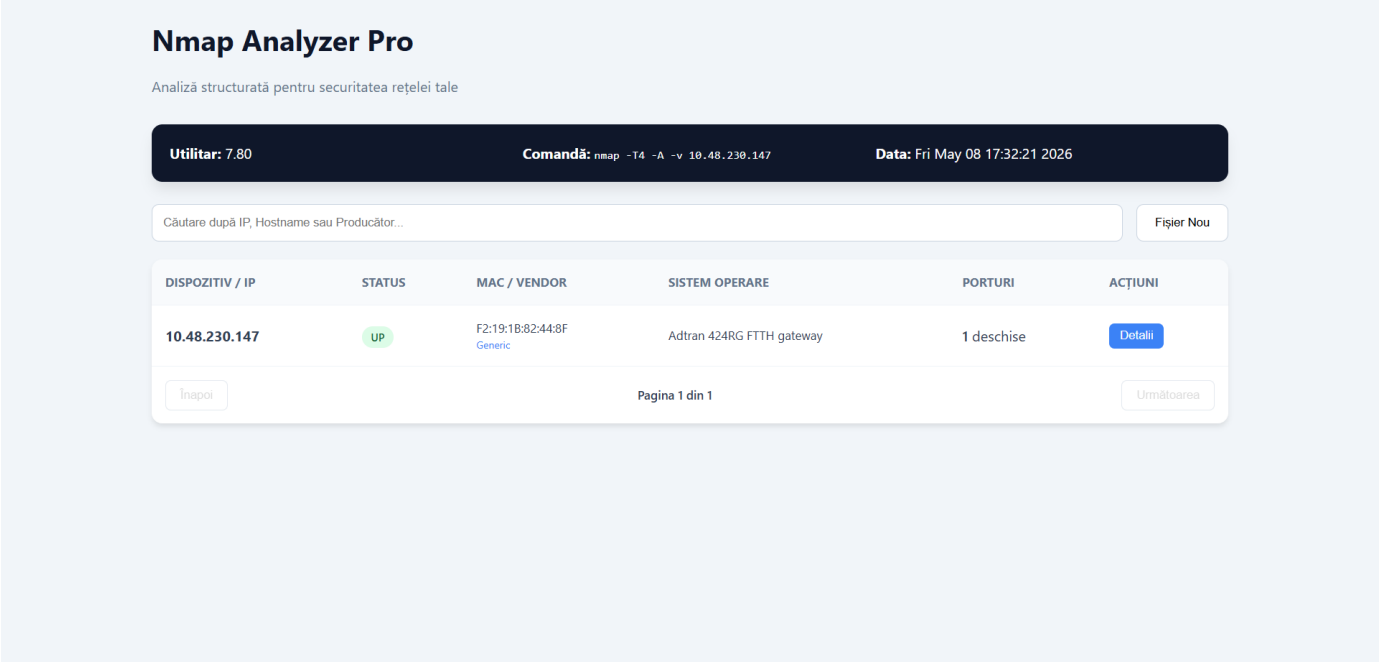
```
PORT {  
  
    string id "Numărul portului"  
  
    string proto "Protocol (tcp/udp)"  
  
    string state "Stare (open/filtered/closed)"  
  
    string service "Nume serviciu (ex: http, ssh)"  
  
    string version "Versiunea software-ului"  
  
}
```

Explicația relațiilor:

- **ScanInfo -> Host (1 la N):** O singură rulare de Nmap (SCANINFO) conține și raportează informații pentru mai multe dispozitive detectate (HOST).
- **Host -> Port (1 la N):** Un singur dispozitiv (HOST) poate avea multiple porturi scanate și raportate (PORT). Toate detaliile despre servicii aparțin de un singur host specific.

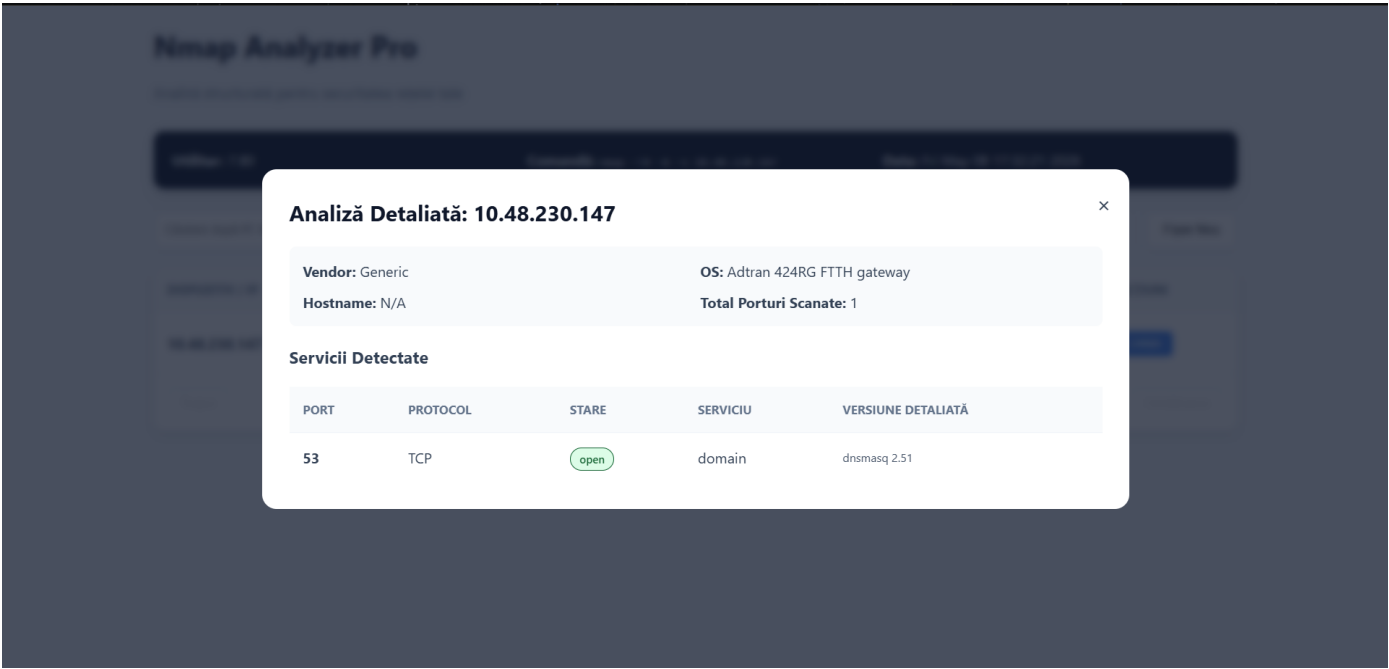
5. Cazuri de Utilizare Complexe.

(Aici trebuie să introduci imaginile în documentul tău final. Pentru a le obține, rulează site-ul tău în browser, încarcă un fișier XML generat de nmap și capturează următoarele scenarii:)



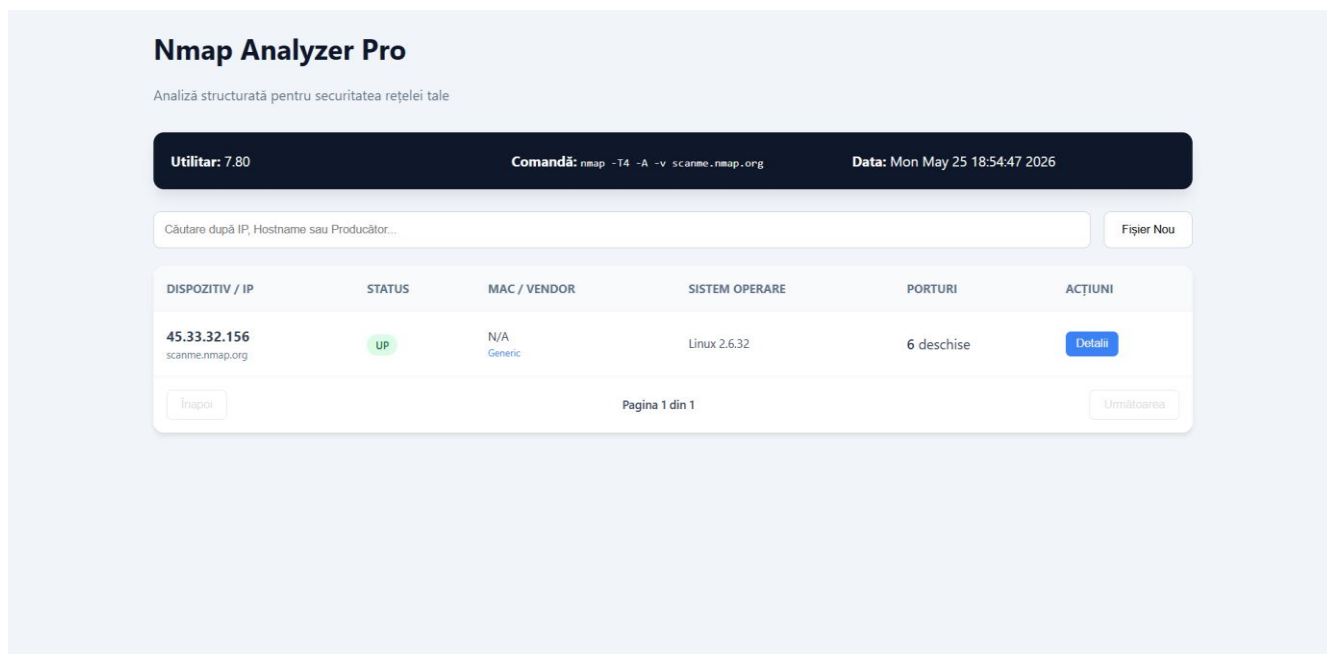
Screenshot 1: Parsarea, Afișarea și Filtrarea datelor asincron

- **Explicație Cumprizătoare:** Acest ecran demonstrează modul în care aplicația gestionează eficient un număr mare de noduri, permițând inginerului de rețea să localizeze instantaneu un anumit producător sau un anumit IP din sute de rânduri, totul rulând fără latență datorită căutării implementate pe client.

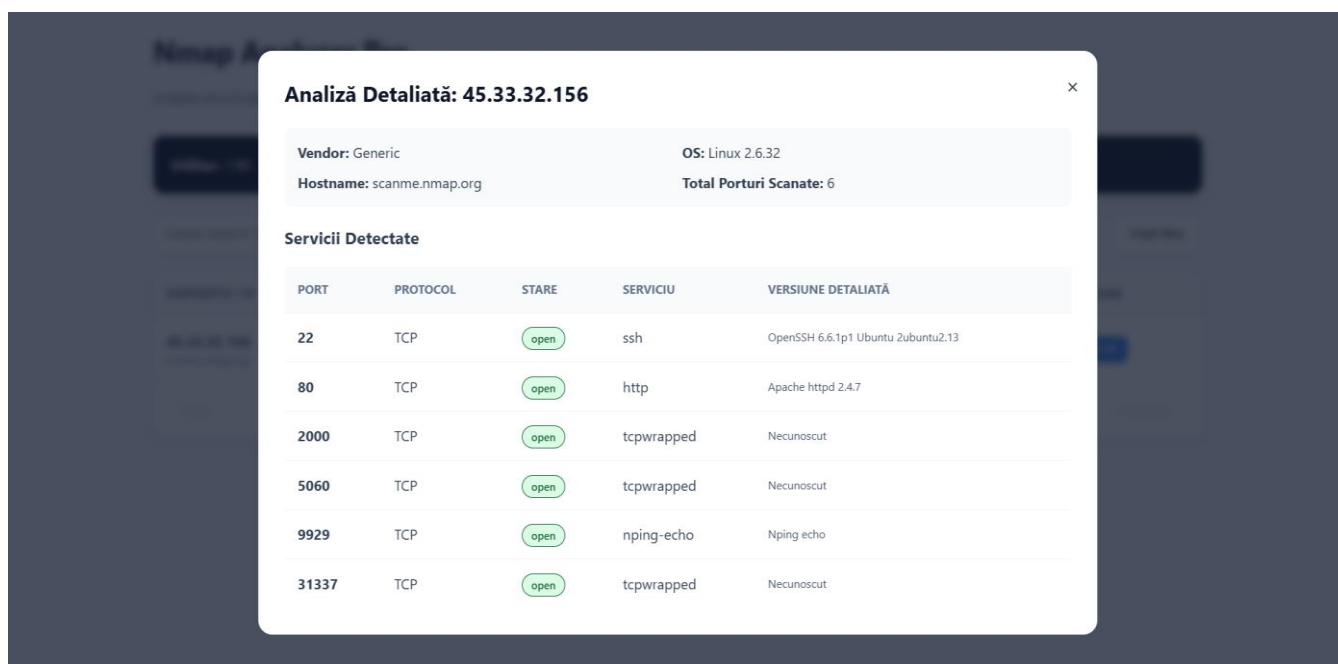


Screenshot 2: Inspecția detaliată a vulnerabilităților și serviciilor

- **Explicație Cumprizătoare:** Această imagine ilustrează capacitatea de *drill-down* a arhitecturii. La cerere, utilizatorul poate accesa array-ul imbricat de ports pentru un dispozitiv specific, vizualizând starea individuală a porturilor și versiunile exacte ale serviciilor software rulate, esențiale pentru identificarea potențialelor vulnerabilități (CVE-uri).



Screenshot 1: Parsarea, Afișarea și Filtrarea datelor asincron



Screenshot 2: Inspecția detaliată a vulnerabilităților și serviciilor